

ecommerce-microservice

Run the stack locally, verify it end-to-end, and exercise a real Stripe test-mode payment — plus the new Kafka UI.

Contents

1. Prerequisites
2. 1. Start infrastructure (Docker)
3. 2. Kafka UI
4. 3. Start the 8 services locally
5. 4. Smoke-test the app
6. 5. Quick payment sanity check (no real Stripe needed)
7. 6. Full real payment test (Stripe CLI, test-mode)
8. 7. Tearing everything down
9. 8. Troubleshooting

Prerequisites

Java	17+ (JDK)
Maven	any recent 3.x (<code>mvn -v</code>)
Docker	Docker Desktop / Docker Engine with Compose v2 (<code>docker compose version</code>)
Node.js	only needed if you also want to run the frontend (<code>cd frontend && npm run dev</code>)
Stripe CLI	only needed for the <i>full</i> real payment test — docs.stripe.com/stripe-cli , then <code>stripe login</code>

Only infrastructure that's painful to run locally is dockerized: Zipkin, Kafka, Kafka UI, and the 4 per-service Postgres databases. The 8 Spring Boot services run as plain local JVMs via `mvn spring-boot:run` — this makes debugging, breakpoints, and hot-reload straightforward.

1 Start infrastructure (Docker)

From the repo root:

```
cp .env.example .env # fill in JWT_SECRET, STRIPE_SECRET_KEY,
STRIPE_WEBHOOK_SECRET_KEY, POSTGRES_PASSWORD
docker compose up -d
```

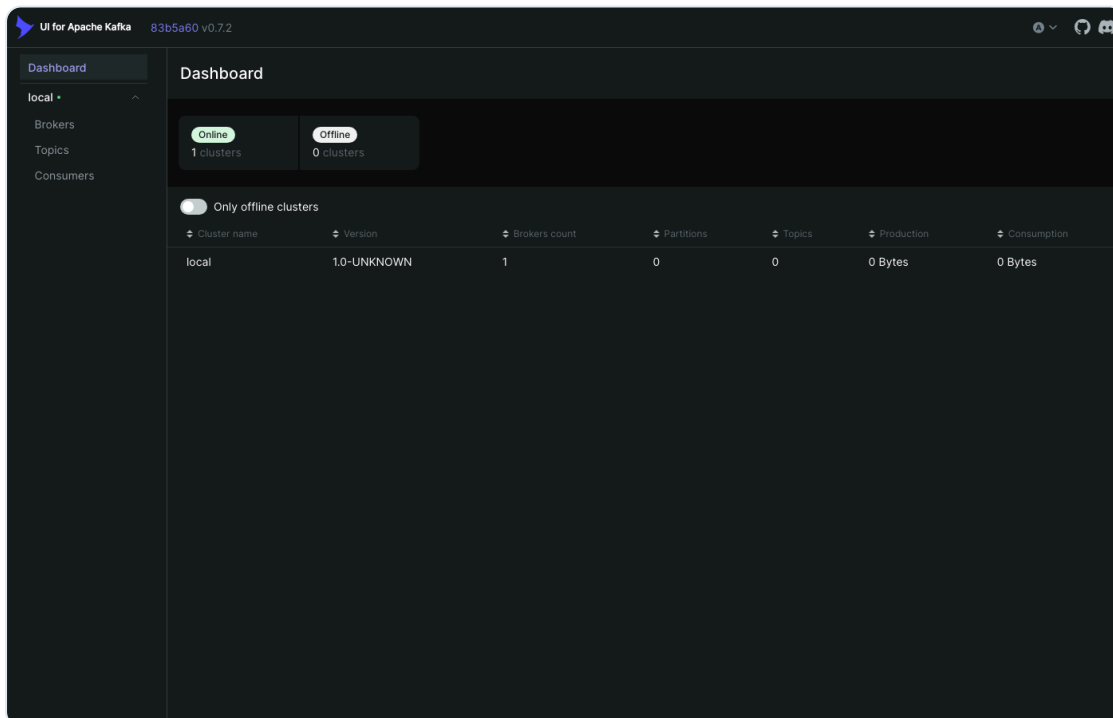
This starts 7 containers:

Container	Purpose	Port
zipkin	distributed tracing UI	9411
kafka	event broker (KRaft mode, no Zookeeper)	9092 (host), 29092 (internal)
kafka-ui	web UI for browsing topics/messages	8090
user-db	Postgres for user-service	5433
catalog-db	Postgres for catalog-service	5434
cart-db	Postgres for cart-service	5435
order-db	Postgres for order-service	5436

Why Kafka has two listeners: Kafka tells clients where to reconnect via its "advertised" address. Host-run services (`order-service` , `payment-service`) need to reach it at `localhost:9092` ; other containers (`kafka-ui`) need to reach it at `kafka:29092` over the compose network. One listener can't correctly serve both audiences, so `docker-compose.yml` defines `PLAINTEXT` (host, :9092) and `INTERNAL` (containers, :29092) side by side.

2 Kafka UI

Once infra is up, open `http://localhost:8090` . You'll see the `local` cluster online, with 0 topics until services start creating/using them.



Once `order-service` and `payment-service` are running (next section) and you've triggered a checkout, use **Topics** → `payment-events` → **Messages** to watch the actual `PaymentEvent` JSON

(`{"orderId":..., "status": "PAID"}`) that `payment-service` published and `order-service` consumed — this is the easiest way to see the async hop in the system happen live.

3 Start the 8 services locally

Order matters: `eureka-server` → `config-server` → everything else. Each service's `application.yml` already defaults to `localhost` for every dependency, matching the ports Docker Compose exposes.

```
set -a; source .env; set +a # exports JWT_SECRET, STRIPE_SECRET_KEY, etc.

mvn -f services/eureka-server/pom.xml spring-boot:run &
mvn -f services/config-server/pom.xml spring-boot:run &
# wait for both to print "Started ...Application", then:
mvn -f services/user-service/pom.xml spring-boot:run &
mvn -f services/catalog-service/pom.xml spring-boot:run &
mvn -f services/cart-service/pom.xml spring-boot:run &
mvn -f services/order-service/pom.xml spring-boot:run &
mvn -f services/payment-service/pom.xml spring-boot:run &
mvn -f services/api-gateway/pom.xml spring-boot:run &
```

Useful URLs once everything is up:

App / API gateway	<code>http://localhost:8080</code>
Eureka dashboard	<code>http://localhost:8761</code>
Config server	<code>http://localhost:8888/{service-name}/default</code>
Zipkin	<code>http://localhost:9411</code>
Kafka UI	<code>http://localhost:8090</code>

Right after startup, the gateway's internal load-balancer cache can lag a few seconds behind Eureka's actual registrations — a request in that window may return `503`. If that happens, just retry a couple of seconds later; no restart needed.

4 Smoke-test the app

```
# Confirm the gateway can reach catalog-service
curl http://localhost:8080/api/products

# Confirm every service registered with Eureka (expect 7 entries)
curl -s http://localhost:8761/eureka/apps -H "Accept: application/json" \
  | python3 -c "import json,sys; d=json.load(sys.stdin); print([a['name'] for a in d['applications']['application']])"
```

A Postman collection covering every endpoint (auth, catalog, cart, checkout, payments) is at `docs/ecommerce-microservice.postman_collection.json` — import it into Postman for interactive

testing, or run it headlessly with `newman run docs/ecommerce-microservice.postman_collection.json --env-var baseUrl=http://localhost:8080`.

5 Quick payment sanity check (no real Stripe needed)

With the dummy keys from `.env.example`, you can verify the checkout flow is wired correctly end-to-end up to the Stripe API boundary, without any Stripe account:

```
TOKEN=$(curl -s -X POST http://localhost:8080/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"you@example.com","password":"..."}' | python3 -c "import
json,sys;print(json.load(sys.stdin)['token'])")

CART_ID=$(curl -s -X POST http://localhost:8080/api/carts | python3 -c "import
json,sys;print(json.load(sys.stdin)['id'])")
curl -s -X POST http://localhost:8080/api/carts/$CART_ID/items -H "Content-Type:
application/json" -d '{"productId":1}'

curl -s -X POST http://localhost:8080/api/checkout \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"cartId":"$CART_ID"}"
```

With a dummy `STRIPE_SECRET_KEY`, this correctly fails when payment-service calls the real Stripe API — order-service catches that and rolls back the order it just created. That failure *is* the expected, correct behavior in this mode; it proves the Feign call chain (gateway → order-service → payment-service) and the rollback-on-failure logic both work.

6 Full real payment test (Stripe CLI, test mode)

This exercises the entire real path: a genuine Stripe Checkout Session, a real (test-mode) card payment, a signed webhook delivered back to `payment-service`, a Kafka event, and `order-service` updating the order to `PAID`.

6.1 — Log in to the Stripe CLI (test mode)

```
stripe login
stripe config --list # note the "test_mode_api_key" (sk_test_...) for the next step
```

6.2 — Start the webhook forwarder

```
stripe listen --forward-to http://localhost:8080/api/checkout/webhook
```

Copy the `whsec_...` secret it prints — you'll need it below. Leave this command running in its own terminal.

`STRIPE_WEBHOOK_SECRET_KEY` is *ephemeral*: `stripe listen` generates a new `whsec_...` value every time you start it (unless you set up a fixed webhook endpoint in the Stripe dashboard instead).

Whichever way you configure it below, you'll need to update this one value each time you restart `stripe listen`.

6.3 — Put the real keys in `.env`

Edit `.env` directly and replace the two dummy values:

```
STRIPE_SECRET_KEY=sk_test_... # from `stripe config --list`  
STRIPE_WEBHOOK_SECRET_KEY=whsec_... # from `stripe listen` above
```

Since `.env` is gitignored and every service already sources it via `set -a; source .env; set +a` before starting, this is all you need — restart (or start) `payment-service` normally and it picks up the real keys with no extra flags:

```
mvn -f services/payment-service/pom.xml spring-boot:run
```

Prefer not to touch `.env` for a one-off test? You can override just for that shell instead — but the order matters:

```
# kill any already-running payment-service first, then:  
set -a; source .env; set +a  
export STRIPE_SECRET_KEY=sk_test_... # override AFTER sourcing .env, not before  
export STRIPE_WEBHOOK_SECRET_KEY=whsec_... # same here  
mvn -f services/payment-service/pom.xml spring-boot:run
```

If you export *before* sourcing `.env`, the dummy values in `.env` silently clobber your overrides when it's sourced, and every webhook will fail signature verification with a generic 500 — exporting after sourcing avoids this entirely, or just skip the whole issue by editing `.env` as in 6.3.

6.4 — Check out

Register/login, build a cart, and hit checkout exactly as in the quick check above. This time the response contains a real Stripe-hosted `checkoutUrl` (starts with `https://checkout.stripe.com/...`) and the order actually persists.

6.5 — Pay with a Stripe test card

Open the `checkoutUrl` in a browser and pay with any Stripe test card, e.g.:

Card number	Expiry	CVC	Result
4242 4242 4242 4242	any future date	any 3 digits	✓ succeeds

On success you're redirected to `http://localhost:8080/checkout-success?orderId=...`.

6.6 — Verify the webhook and the resulting order status

```
# The stripe listen terminal should show:
# --> payment_intent.succeeded [...]
# <-- [200] POST http://localhost:8080/api/checkout/webhook [...]

curl -s http://localhost:8080/api/orders/<orderId> -H "Authorization: Bearer $TOKEN"
# {"id":..., "status": "PAID", ...}
```

You can also watch this land in real time in **Kafka UI** → **Topics** → **payment-events** → **Messages** (see section 2).

7 Tearing everything down

```
# stop the 8 local services (Ctrl+C each, or):
kill -f "spring-boot:run"

# stop stripe listen if you ran the full payment test:
kill -f "stripe listen"

# stop infra:
docker compose down
```

8. Troubleshooting

Symptom	Likely cause / fix
Gateway returns 503 right after startup	Load-balancer cache hasn't refreshed yet — retry in a few seconds.
Webhook always returns 500 "Error creating a checkout session"	Signature verification failed — usually a stale/wrong <code>STRIPE_WEBHOOK_SECRET_KEY</code> . Re-check the value from the currently-running <code>stripe listen</code> session (it changes each time you restart <code>stripe listen</code>), and make sure you exported it <i>after</i> sourcing <code>.env</code> .
Checkout succeeds but order stays PENDING	Give the Kafka consumer a couple of seconds, or check <code>payment-events</code> in Kafka UI to confirm the event was actually published.
Kafka UI shows the cluster offline	Confirm <code>kafka-ui</code> 's <code>KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS</code> is <code>kafka:29092</code> (the INTERNAL listener), not <code>localhost:9092</code> — the latter only resolves for host processes, not other containers.
"No servers available for service: X" in gateway logs	That service hasn't registered with Eureka yet, or crashed on startup — check its own log and <code>http://localhost:8761</code> .

Generated for the ecommerce-microservice project. See README.md for architecture details, and docs/system-design.drawio / docs/ecommerce-microservice.postman_collection.json for the diagram and API collection.